

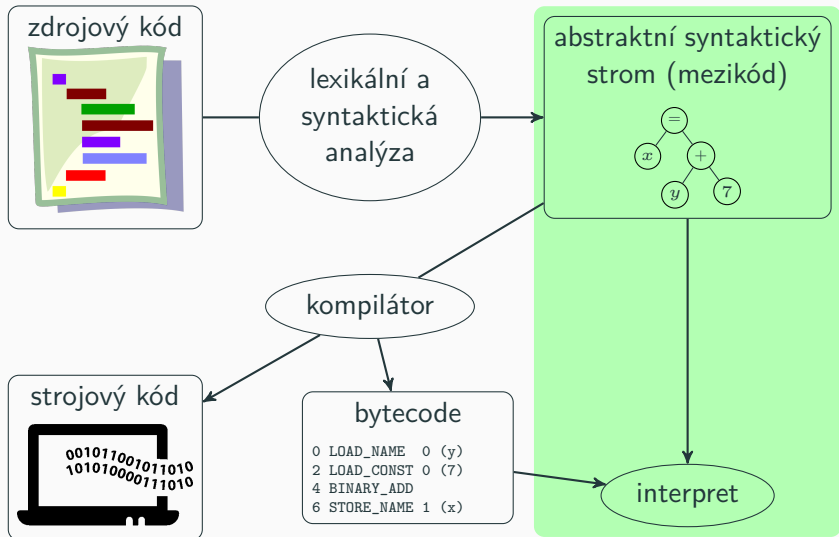
Interpret jednoduchého programovacího jazyka

IB111 ZÁKLADY PROGRAMOVÁNÍ

Tomáš Vojnar (na základě slajdů Nikoly Beneše)

5. prosince 2025

Zpracování programů (zjednodušeně)



- Syntax co nejvíc podobná Pythonu.
- Jediný datový typ: celé číslo.
- Aritmetické operace: +, -, *, / (celočíselné dělení).
- Operátory porovnání (==, < apod.), logické výrazy.
- Příkazy:
 - přiřazení,
 - volně stojící výrazy (zejména tedy i volání funkce),
 - `if` / `elif` / `else`,
 - `while`,
 - `return`.
- Hlavní program jen definice funkcí:
 - předpokládáme, že funkce vždy vrátí celé číslo,
 - spuštění programu spustí funkci `main`,
 - speciální vestavěná funkce `print`.

1. Jednoduchá kalkulačka:
 - číselné konstanty, aritmetické operace.
2. Kalkulačka s proměnnými:
 - proměnné, přiřazení.
3. Programy bez větvení:
 - jen posloupnosti přiřazení.
4. Programy s větvením:
 - logické výrazy, `if`, `while`.
5. Programy s funkcemi.

Reprezentace výrazů

Expression = int | Op

- Konstanty: čísla typu `int`.
- Operace:

```
class Op:
    def __init__(self, op: str,
                  operands: list['Expression']):
        self.op = op
        self.operands = operands
```

Poznámka: Abychom odlišili unární plus a minus, použijeme pro jejich reprezentaci řetězce `'+u'` a `'-u'`.

```
def eval_expr(expr: Expression) -> int:
    if isinstance(expr, int):
        return expr

    # Op
    ops = [eval_expr(op) for op in expr.operands]

    if expr.op == '-u':
        return -ops[0]

    if expr.op == '+':
        return ops[0] + ops[1]

    # ...
```

```
# ...
```

```
if expr.op == '/':  
    return ops[0] // ops[1]
```

```
assert False # unreachable
```

- Co když dojde k dělení nulou?

Zpracování chyb

- Vyhodnocení vrátí buď hodnotu nebo objekt typu `Error`.
- Pokud je nějaká část výrazu chybná, je chybný celý výraz.

```
class Error:  
    def __init__(self, msg: str):  
        self.msg = msg
```

```
ops = []
for op in expr.operands:
    value = eval_expr(op)
    if isinstance(value, Error):
        return value
    ops.append(value)

# ...

if expr.op == '/':
    if ops[1] == 0:
        return Error("Division by zero.")

    return ops[0] // ops[1]
```



```
def parse_expr(line: str) -> Expression | Error:
```

- Provádí syntaktickou analýzu.
- Z výrazu zapsaného v řetězci `line` vytvoří syntaktický strom (objekt typu `Expression`) nebo reportuje syntaktickou chybu.

```
def draw(expr: Expression, filename: str) -> None:
```

- Zadaný syntaktický strom vykreslí do zadaného souboru ve formátu GraphViz (<http://webgraphviz.org>).

```
def main() -> None:
```

- Jednoduchá kalkulačka.
- Načítá výrazy po řádcích a vypisuje jejich hodnotu.

Reprezentace výrazů

Expression = `int` | `str` | `Op`

- Konstanty: čísla typu `int`.
- Proměnné: reprezentovány řetězci.
- Operace: typ `Op` jako v předchozím.

Vyhodnocení výrazů

- Je závislé na tzv. *prostředí* (environment):
 - přiřazuje proměnným jejich aktuální hodnotu,
 - v našem jednoduchém příkladu: `dict[str, int]`.

```
def eval_expr(expr: Expression,
              env: dict[str, int]) -> int | Error:
    # ...

    if isinstance(expr, str):
        if expr not in env:
            return Error("Variable <" + expr +
                          "> has no value.")
        return env[expr]

    # Op
    ops = []
    for op in expr.operands:
        value = eval_expr(op, env)
    # ...
```

Program

- Posloupnost přiřazení ve tvaru `proměnná = výraz`.
- Vykonání programu – postupné vyhodnocení přiřazení.
- Výsledek: hodnoty jednotlivých proměnných.

```
class Assignment:
    def __init__(self, var: str, expr: Expression):
        self.var = var
        self.expr = expr
```

```
Program = list[Assignment]
```

```
class Interpreter:
    def __init__(self, program: Program):
        self.program = program
        self.env: dict[str, int] = {}
```

- Metoda eval vyhodnocuje výrazy.
- Metoda execute vykoná jeden příkaz (přiřazení):
 - vrací `None` nebo `Error`.
- Metoda run spustí program:
 - vykonává postupně všechny příkazy,
 - dojde-li k chybě, tak ji vrátí,
 - je-li vše OK, vrátí přiřazení hodnot proměnným.

Spuštění z příkazové řádky

- Se jménem souboru s programem: vykoná program.
- S `-d` před jménem souboru: vykreslí AST programu do `ast.dot`.

Logické hodnoty

- Hodnotu *nepravda* budeme reprezentovat číslem 0.
- Hodnotu *pravda* libovolným nenulovým číslem:
 - budeme-li mít volbu, použijeme číslo 1.

Operátory porovnávání

```
if expr.op == '==':  
    return 1 if ops[0] == ops[1] else 0
```

```
if expr.op == '!=':  
    return 1 if ops[0] != ops[1] else 0
```

```
# ...
```

Operátor `not`

```
if expr.op == 'not':  
    return 1 if ops[0] == 0 else 0
```

Operátory `and` a `or`

- Mohli bychom je implementovat jako ostatní operátory:
 - vyhodnotí se vždy oba podvýrazy,
 - to dost často *nechceme*:

```
count != 0 and total / count > 1
```

- Použijeme *zkrácené vyhodnocování*:
 - vyhodnotíme nejprve levý podvýraz a teprve je-li to potřeba, vyhodnotíme i ten pravý.

```
if expr.op == 'and':
    left = self.eval(expr.operands[0])
    if isinstance(left, Error) or left == 0:
        return left
    return self.eval(expr.operands[1])

if expr.op == 'or':
    left = self.eval(expr.operands[0])
    if isinstance(left, Error) or left != 0:
        return left
    return self.eval(expr.operands[1])

ops = []
for op in expr.operands:
    # ...
```


- O něco hezčí řešení:

```
if expr.op in ['and', 'or']:
    left = self.eval(expr.operands[0])
    if shortcut(left, expr.op):
        return left
    return self.eval(expr.operands[1])

# ...
```

- Pomocná funkce:

```
def shortcut(left: int | Error, op: str) -> bool:
    return isinstance(left, Error) or (
        left == 0 if op == 'and' else left != 0
    )
```

```
class While:
    def __init__(self, cond: Expression,
                  body: 'Block'):
        self.cond = cond
        self.body = body

class If:
    def __init__(self, cond: Expression,
                  main_body: 'Block'):
        self.cond = cond
        self.main_body = main_body
        self.else_body: Block | None = None
```

```
Statement = Assignment | If | While
```

```
Block = list[Statement]
```

```
def exec_block(self, block: Block) \
    -> None | Error:
```

- Vykonává příkazy v bloku, zastaví na první chybě.

```
def execute(self, statement: Statement) \
    -> None | Error:
```

```
if isinstance(statement, If):
    return self.exec_if(statement)
```

```
if isinstance(statement, While):
    return self.exec_while(statement)
```

```
# Assignment
```

```
value = self.eval(statement.expr)
```

```
# ...
```

```
def exec_if(self, if_stmt: If) -> None | Error:
    cond = self.eval(if_stmt.cond)
    if isinstance(cond, Error):
        return cond

    if cond != 0:
        return self.exec_block(if_stmt.main_body)

    if if_stmt.else_body is not None:
        return self.exec_block(if_stmt.else_body)

    return None
```

```
def exec_while(self, while_stmt: While) \
    -> None | Error:
    while True:
        cond = self.eval(while_stmt.cond)
        if isinstance(cond, Error):
            return cond

        if cond == 0:
            return None

        block = self.exec_block(while_stmt.body)
        if isinstance(block, Error):
            return block
```

```
def pow(base, exp):
```

formální parametry

```
    result = 1
```

```
    while exp > 0:
```

```
        exp = exp - 1
```

```
        result = result * base
```

```
    return result
```

tělo funkce

```
class Return:
```

```
    def __init__(self, expr: Expression):
```

```
        self.expr = expr
```

```
class Function:
```

```
    def __init__(self, params: list[str], body: Block):
```

```
        self.params = params
```

```
        self.body = body
```

```
def main():  
    return pow(2, 2 * 10)
```

- Volání funkce je výraz.
- Jméno funkce, skutečné parametry (argumenty) – opět výrazy.

Expression = int | str | Op | Call

```
class Call:  
    def __init__(self, fun: str,  
                  args: list['Expression']):  
        self.fun = fun  
        self.args = args
```

```
Program = dict[str, Function]
```

- Program je slovník funkcí.
- Spuštění programu je vykonání funkce `main`.

Místo jednoho prostředí, které přiřazuje hodnoty proměnným, máme *zásobník* obsahují tzv. *rámce* (aktivační záznamy funkcí).

- Každý rámec přiřazuje hodnoty lokálním proměnným v aktuálním vykonávání funkce.

```
Frame = dict[str, int]
```

```
class Interpreter:
```

```
    def __init__(self, program: Program):  
        self.program = program  
        self.stack: list[Frame] = []
```


- Potřebujeme umět přerušit vykonávání funkce při provedení příkazu `return`.
- Řešení: metody `execute`, `exec_block` apod. nebudou vracet `None` | `Error`, ale `None` | `int` | `Error`
 - `None` znamená, že příkaz/blok příkazů se vykonal celý bez provedení `return` (a bez chyb).
 - Číselná hodnota znamená, že došlo k provedení `return` s výrazem, který se vyhodnotil na dané číslo.
 - Objekt `Error` reprezentuje běhovou chybu (jako předtím).

```
def execute(self, statement: Statement) \
    -> None | int | Error:
    if isinstance(statement, If):
        return self.exec_if(statement)
    if isinstance(statement, While):
        return self.exec_while(statement)
    if isinstance(statement, Return):
        return self.eval(statement.expr)

    # Assignment
    value = self.eval(statement.expr)
    if isinstance(value, Error):
        return value

    self.stack[-1][statement.var] = value
    return None
```

- Detaily metod `exec_...` viz přiložený zdrojový kód.

```
def eval_call(self, call: Call) -> int | Error:
```

- Ověříme, zda je funkce s daným jménem `call.fun` definovaná – tj. zda je ve slovníku `program`.
- Ověříme, zda sedí počet formálních parametrů funkce s počtem argumentů volání.
- Vyhodnotíme (pomocí `eval`) všechny argumenty.
 - Vrátíme `Error`, pokud někde nastane chyba.
- Vytvoříme na zásobníku nový rámec, v němž jménům formálních parametrů přiřadíme hodnoty argumentů.
- Vykonáme tělo funkce (pomocí `exec_block`) a zapamatujeme si případnou návratovou hodnotu.
- Odstraníme rámec z vrcholu zásobníku:
 - Po skončení těla funkce lokální proměnné přestaly existovat.
- Vrátíme návratovou hodnotu.
 - Pokud vykonání funkce neskončilo příkazem `return`, vrátíme 0.

```
Statement = \  
    Assignment | Expression | If | While | Return
```

- Libovolný volně stojící výraz je příkaz (tak jako v Pythonu).¹
 - Užitečné, pokud máme nějaké výrazy s vedlejšími efekty.
- (Nechybovou) návratovou hodnotu volání *ignorujeme*.

```
def execute(self, statement: Statement) \  
    -> None | int | Error:  
    # ...  
  
    # Expression statement  
    result = self.eval(statement)  
    if isinstance(result, Error):  
        return result  
    return None
```

¹Ale pozor: pořád platí, že příkaz není výraz.

```
def eval_call(self, call: Call) -> int | Error:

    # ...

if name == 'print':
    # Special hard-coded function.
    sep = ""
    for arg in args:
        sys.stdout.write(sep + to_str(arg))
        sep = " "
    sys.stdout.write("\n")
    return 0

    # ...
```

- Chceme k reportu chyby přidat:
 - jméno funkce,
 - řádek, na kterém chyba vznikla,
 - hodnoty lokálních proměnných,
 - místo, odkud byla funkce volána atd. (stack trace).
- K tomu účelu rozšíříme rámce na zásobníku.

```
class Frame:
```

```
    def __init__(self, fun: str):  
        self.fun = fun  
        self.line_no = 0  
        self.env: dict[str, int] = {}
```

- Další úpravy – viz přiložený zdrojový kód.

Máme interpret jednoduchého jazyka podobného Pythonu, který má:

- proměnné celočíselného typu,
- aritmetické a logické výrazy,
- řídicí struktury **if** a **while**,
- volání funkcí (včetně rekurze).